Express.js — Framework Overview, Installation & App Creation

Section: 7 — Express.js Framework Basics

Subtopic: 01 — Express Introduction

Estimated Duration: 3–4 hours

Theory Practical

Topics covered: What Express is and why it exists, minimalist middleware-based architecture, installing Express via NPM, creating an application with `express()`, listening on ports with `app.listen()`, sending basic responses, understanding the request/response cycle in Express

Learning Objectives — This Subtopic

- Explain what Express.js is, its role in the Node.js ecosystem, and why it exists on top of the core `http` module
- Install Express into a Node.js project and set up a minimal application structure
- Create an Express application instance using `express()` and bind it to a network port
- Define basic route handlers that respond to HTTP GET requests
- Send different response types (plain text, HTML, JSON) using Express's response methods
- Contrast raw `http.createServer()` code with equivalent Express code to appreciate the abstraction

1. Why Express Exists

In Section 6, you built HTTP servers using Node's core `http` module. That code worked, but it required manual effort for everything: parsing URLs, checking HTTP methods, setting headers, sending responses. For a server with dozens of endpoints, the code becomes repetitive and fragile.

Express.js is a **minimal and unopinionated web framework** for Node.js that provides a thin layer of abstraction over the core `http` module. It handles the repetitive plumbing so you can focus on application logic.

Here is a concrete comparison. Recall the kind of code required to handle two routes with the raw `http` module:

raw-http-example.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Home page');
  } else if (req.method === 'GET' && req.url === '/about') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('About page');
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('Not found');
  }
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Now the same thing with Express:

express-example.js

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
```

```
  res.send('Home page');
});

app.get('/about', (req, res) => {
  res.send('About page');
});

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

The Express version is shorter, more readable, and — critically — scales better. Adding a new route is a single `app.get()` call rather than another branch in a growing `if/else` chain. Express also handles the 404 case automatically, sets appropriate headers, and provides dozens of convenience methods on the request and response objects.

> **Key Insight:** Express does not replace the `http` module. It wraps it. Under the hood, `app.listen()` calls `http.createServer()`. Everything you learnt in Section 6 still applies — Express simply provides a better interface for the same underlying mechanics.

1.1 What "Minimalist and Unopinionated" Means

Express gives you routing, middleware, and a request/response API. It does *not* prescribe how you structure your application, which template engine you use, or which database you connect to. This is what "unopinionated" means — decisions about architecture are yours to make. Compare this with frameworks like Ruby on Rails or Django, which enforce specific project structures and conventions.

The "minimalist" part means Express's core is deliberately small. Features like body parsing, session management, and authentication are handled by separate middleware packages that you add as needed.

1.2 Express in the Ecosystem

Express is the most widely used Node.js web framework. It has been in production since 2010 and underpins a large portion of the Node.js server ecosystem. Many higher-level frameworks (NestJS, Sails.js, LoopBack) are built on top of Express.

> **Analogy:** Think of the raw `http` module as driving a manual-transmission car — you have full control, but you manage every gear change yourself. Express is an automatic transmission — it handles the routine shifting for you while still letting you override when you need to. The engine (Node.js) is exactly the same in both cases.

---

2. Project Setup and Installation

Before writing any Express code, you need a properly initialised Node.js project. This follows the same workflow covered in Section 2 (NPM Fundamentals).

2.1 Create the Project Directory

Open your terminal and create a new project folder:

```
mkdir express-intro && cd express-intro
```

Initialise a `package.json`:

```
npm init -y
```

> **Output:**
>
> ```
> Wrote to /Users/you/express-intro/package.json:
>
> {
>   "name": "express-intro",
>   "version": "1.0.0",
>   "description": "",
>   "main": "index.js",
>   "scripts": {
>     "test": "echo \"Error: no test specified\" && exit 1"
>   },
>   "keywords": [],
>   "author": "",
>   "license": "ISC"
> }
> ```

2.2 Install Express

Install Express as a project dependency:

```
npm install express
```

Output:

```
added 64 packages, and audited 65 packages in 3s

13 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

This adds Express to the `dependencies` section of your `package.json` and downloads it (along with its own dependencies) into `node_modules/`.

Verify the installation by checking the installed version:

```
npm list express
```

Output:

```
[email protected] /Users/you/express-intro
└── [email protected]
```

2.3 Project Structure

At this point your project looks like this:

```
express-intro/
├── node_modules/
├── package.json
└── package-lock.json
```

You will create your application files in the root of this directory. As the project grows in later sections, you will organise code into folders — but for now, a single file is sufficient.

> **Note:** Add a `start` script to your `package.json` so you can run your server with `npm start` instead of typing the full `node` command each time. Open `package.json` and change the `"scripts"` section:
>
> ```
> "scripts": {
>   "start": "node index.js"
> }
> ```

---

3. Creating Your First Express Application

An Express application is created by calling the `express()` function. This returns an application object (conventionally named `app`) that has methods for routing, configuring middleware, and starting the server.

Stage 1 The Absolute Minimum

Create a file called `index.js` in your project root:

index.js

```
const express = require('express');
const app = express();

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

Run it:

```
node index.js
```

> Output:

```
Server is running on http://localhost:3000
```

What is happening:

- `require('express')` imports the Express module. It returns a function.
- `express()` calls that function, which creates a new Express application instance.
- `app.listen(3000, callback)` binds the application to TCP port 3000 and starts listening for incoming HTTP connections. The callback fires once the server is ready.

If you open `http://localhost:3000` in a browser right now, you will see **"Cannot GET /"**. This is Express's default response when no route matches the request. The server is running — it simply has no routes defined yet.

Press `Ctrl` + `C` in the terminal to stop the server before continuing.

Stage 2 Adding a Route

A **route** tells Express: "When a request arrives with this HTTP method and this URL path, run this function." The simplest route handles a GET request to the root path (`/`):

index.js

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello, Express!');
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

Run it:

```
node index.js
```

Output:

```
Server is running on http://localhost:3000
```

Now open `http://localhost:3000` in your browser:

Output:

```
Hello, Express!
```

What is happening:

- `app.get(path, handler)` registers a route handler for HTTP GET requests to the given path.
- The handler function receives two objects: `req` (the incoming request) and `res` (the outgoing response).
- `res.send()` sends a response body and automatically sets the `Content-Type` header based on the argument type. A string argument defaults to `text/html`.

> **Key Insight:** Unlike the raw `http` module where you must call `res.writeHead()` and `res.end()` separately, `res.send()` handles status code, headers, and body termination in a single call. This is one of Express's core conveniences.

---

4. The `app` Object in Detail

The object returned by `express()` is the central piece of every Express application. It is responsible for:

- **Routing:** Mapping HTTP method + URL path combinations to handler functions
- **Middleware:** Running functions in sequence on every request (covered in Section 7, subtopic 03)
- **Configuration:** Storing application-level settings
- **Server lifecycle:** Starting and stopping the HTTP server

4.1 `app.listen()` — How It Works

`app.listen()` is a convenience method. Internally, it does this:

what-listen-does.js

```
const express = require('express');
const http = require('http');
const app = express();

app.get('/', (req, res) => {
  res.send('Created with http.createServer');
});

// This is what app.listen() does internally:
const server = http.createServer(app);
server.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

Run it:

```
node what-listen-does.js
```

> Output:
>
> ```
> Server is running on http://localhost:3000
> ```

Visit `http://localhost:3000`:

> Output:
>
> ```
> Created with http.createServer
> ```

What is happening: The Express `app` is itself a valid request listener function — it has the signature `(req, res)` that `http.createServer()` expects. This means Express is not a replacement for the `http` module; it is a request handler that plugs into it.

You rarely need to call `http.createServer(app)` yourself. The main reasons to do so are when you need direct access to the underlying `http.Server` object (for example, to attach WebSocket listeners) or when setting up HTTPS. For now, `app.listen()` is sufficient.

Important: `app.listen()` returns the `http.Server` instance. If you need a reference to it (e.g., for graceful shutdown), capture it in a variable:

```
const server = app.listen(3000, () => {
  console.log('Running');
});

// Later, for graceful shutdown:
// server.close();
```

4.2 `app.set()` and `app.get()` for Configuration

Express has a built-in settings system accessed via `app.set(name, value)` and `app.get(name)`. Note that `app.get()` has two roles: with one argument it retrieves a setting; with two arguments (path, handler) it registers a GET route.

`app-settings.js`

```
const express = require('express');
const app = express();

// Store a custom setting
app.set('appName', 'My Express App');

// Retrieve it
console.log(app.get('appName'));

app.get('/', (req, res) => {
  res.send('Application: ' + app.get('appName'));
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

`node app-settings.js`

> Output:

```
My Express App
Server is running on http://localhost:3000
```

Visit `http://localhost:3000`:

> **Output:**
>
> ```
> Application: My Express App
> ```

Express also has built-in settings that control framework behaviour. Some useful ones include `'env'` (defaults to `process.env.NODE_ENV` or `'development'`), `'view engine'` (covered in Section 9), and `'json spaces'` (pretty-prints JSON responses when set).

---

5. Response Methods

Express extends the native `http.ServerResponse` object with convenience methods. Here are the three you will use most frequently.

5.1 `res.send()` — General-Purpose Response

`res.send()` is the most versatile response method. It automatically determines the `Content-Type` based on the argument:

res-send.js

```
const express = require('express');
const app = express();

// String argument → Content-Type: text/html
app.get('/text', (req, res) => {
  res.send('Plain text response');
});

// HTML string → Content-Type: text/html
app.get('/html', (req, res) => {
  res.send('<h1>HTML Response</h1><p>This is rendered as HTML.</p>');
});
```

```
// Buffer argument → Content-Type: application/octet-stream
app.get('/buffer', (req, res) => {
  res.send(Buffer.from('Buffer data'));
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

`node res-send.js`

> Output:
>
> `Server is running on http://localhost:3000`

Test each endpoint in a browser or with `curl http://localhost:3000/text`:

> Output:
>
> `Plain text response`

`curl http://localhost:3000/html`

> Output:
>
> `<h1>HTML Response</h1><p>This is rendered as HTML.</p>`

5.2 `res.json()` — JSON Responses

For API endpoints, `res.json()` serialises a JavaScript object or array to JSON and sets `Content-Type: application/json`:

res-json.js

```
const express = require('express');
const app = express();

app.get('/api/status', (req, res) => {
```

```
  res.json({
    status: 'running',
    uptime: process.uptime(),
    timestamp: new Date().toISOString()
  });
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

`node res-json.js`

> Output:
>
> ```
> Server is running on http://localhost:3000
> ```

`curl http://localhost:3000/api/status`

> Output:
>
> ```
> {"status":"running","uptime":2.345,"timestamp":"2025-02-20T14:30:00.000Z"}
> ```

What is happening: `res.json()` calls `JSON.stringify()` on the argument, sets the `Content-Type` header, and sends the response. It also properly handles `null` and `undefined` values. While `res.send()` can also send objects (and will detect them as JSON), using `res.json()` makes your intent explicit.

5.3 `res.status()` — Setting the Status Code

`res.status()` sets the HTTP status code. It returns the response object, so you can chain it with `.send()` or `.json()`:

res-status.js

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.status(200).send('OK');
});
```

```
app.get('/not-here', (req, res) => {
  res.status(404).json({
    error: 'Resource not found',
    path: req.path
  });
});

app.get('/server-error', (req, res) => {
  res.status(500).json({
    error: 'Internal server error'
  });
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

`node res-status.js`

Output:

```
Server is running on http://localhost:3000
```

`curl -i http://localhost:3000/not-here`

Output:

```
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8

{"error":"Resource not found","path":"/not-here"}
```

Note: Without an explicit `res.status()` call, Express defaults to status `200`. Set the status explicitly whenever you return an error or non-standard response.

6. Defining Multiple Routes

A real application needs more than one endpoint. Routes are matched in the order they are defined — Express checks each one from top to bottom and runs the first matching handler.

Stage 3 Building a Multi-Route Application

`multi-routes.js`

```
const express = require('express');
const app = express();

const PORT = 3000;

// Home page
app.get('/', (req, res) => {
  res.send('<h1>Welcome</h1><p>Navigate to /about or /api/info</p>');
});

// About page
app.get('/about', (req, res) => {
  res.send('<h1>About</h1><p>This is a learning project.</p>');
});

// JSON API endpoint
app.get('/api/info', (req, res) => {
  res.json({
    app: 'Express Introduction',
    version: '1.0.0',
    node: process.version
  });
});

// Health check endpoint
app.get('/health', (req, res) => {
  res.status(200).json({ status: 'healthy' });
});

app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

```
node multi-routes.js
```

Output:

```
Server is running on http://localhost:3000
```

```
curl http://localhost:3000/api/info
```

Output:

```
{"app":"Express Introduction","version":"1.0.0","node":"v20.11.0"}
```

```
curl http://localhost:3000/health
```

Output:

```
{"status":"healthy"}
```

What is happening:

- Each `app.get()` registers a handler for a specific path. When a GET request arrives, Express iterates through the registered routes and invokes the first match.
- The `PORT` variable is extracted into a constant. This is a common pattern — it makes it easy to later read the port from an environment variable (`process.env.PORT || 3000`).
- Unmatched requests (e.g., `/contact`) receive Express's default 404 response: "Cannot GET /contact".

Warning: Route order matters. If you define a catch-all route (like `app.get('*', ...)`) before your specific routes, it will intercept every request and the specific routes will never execute. Always define more specific routes first.

---

7. A First Look at the Request Object

The `req` object passed to route handlers is an enhanced version of Node's native `http.IncomingMessage`. Express adds properties that make extracting request information straightforward. Detailed request handling (body parsing, file uploads) is covered in Section 8, but here are the properties available immediately.

req-basics.js

```
const express = require('express');
const app = express();

app.get('/inspect', (req, res) => {
  res.json({
    method: req.method,
    path: req.path,
    url: req.url,
    hostname: req.hostname,
    ip: req.ip,
    protocol: req.protocol,
    query: req.query,
    headers: {
      host: req.headers['host'],
      userAgent: req.headers['user-agent']
    }
  });
});

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

```
node req-basics.js
```

> Output:
>
> ```
> Server is running on http://localhost:3000
> ```

```
curl http://localhost:3000/inspect?name=test&page=1
```

Output:

```
{
  "method": "GET",
  "path": "/inspect",
  "url": "/inspect?name=test&page=1",
  "hostname": "localhost",
  "ip": "::1",
  "protocol": "http",
  "query": {
    "name": "test",
    "page": "1"
  },
  "headers": {
    "host": "localhost:3000",
    "userAgent": "curl/8.4.0"
  }
}
```

What is happening:

- `req.query` automatically parses the URL query string into an object. With the raw `http` module, you had to do this manually with the `url` or `querystring` modules.
- `req.path` gives the URL path without the query string, while `req.url` includes it.
- All query string values are strings — `"1"`, not `1`. You must convert them yourself if you need numbers.

---

8. Using Environment Variables for the Port

Hard-coding the port number works for development, but production environments typically assign ports via environment variables. The standard pattern:

env-port.js

```
const express = require('express');
const app = express();

const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.json({ message: 'Running', port: PORT });
```

```
});

app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

Run with the default port:

```
node env-port.js
```

> **Output:**
>
> ```
> Server is running on http://localhost:3000
> ```

Run with a custom port:

macOS / Linux: `PORT=4000 node env-port.js`
Windows (cmd): `set PORT=4000 && node env-port.js`
Windows (PowerShell): `$env:PORT=4000; node env-port.js`

> **Output:**
>
> ```
> Server is running on http://localhost:4000
> ```

`process.env.PORT || 3000` reads the `PORT` environment variable. If it is not set, the `||` operator falls back to `3000`. This pattern is used in virtually every Express application you will encounter.

Summary

Concept	Key Point
What is Express	A minimal, unopinionated web framework that wraps Node's `http` module
Installation	

Concept	Key Point
	`npm install express` — adds to project dependencies
`express()`	Creates an application instance with routing, middleware, and configuration methods
`app.listen(port, cb)`	Binds the app to a port; wraps `http.createServer(app)` internally
`app.get(path, handler)`	Registers a handler for GET requests to the given path
`res.send()`	Sends a response; auto-detects Content-Type from the argument
`res.json()`	Sends a JSON response; explicitly sets `application/json`
`res.status(code)`	Sets the HTTP status code; chainable with `.send()` or `.json()`
`req.query`	Parsed query string object; values are always strings
Environment port	`process.env.PORT || 3000` — standard pattern for configurable ports

---

Interview Questions

1. What is Express.js and what problem does it solve?
Expected answer: Express is a minimal web framework for Node.js that provides routing, middleware, and an enhanced request/response API on top of the core `http` module. It solves the problem of repetitive boilerplate code (URL parsing, method checking, header management) that you must write manually with the raw `http` module.

2. What does it mean that Express is "unopinionated"?
Expected answer: Express does not enforce a specific project structure, template engine, or ORM. The developer chooses how to organise the application. This contrasts with opinionated frameworks like Rails or Django that prescribe conventions.

3. What happens internally when you call `app.listen(3000)`?
Expected answer: Express creates an `http.Server` using `http.createServer(app)` and calls `.listen(3000)` on it. The Express `app` function serves as the request listener. The method returns the underlying `http.Server` instance.

4. What is the difference between `res.send()` and `res.json()`?
Expected answer: `res.send()` is general-purpose and auto-detects the Content-Type from the argument type (string → text/html, object → application/json, Buffer → application/octet-stream). `res.json()` always serialises the argument as JSON and sets `Content-Type: application/json`. Using `res.json()` makes intent explicit for API responses.

5. Why should you use `process.env.PORT || 3000` instead of hard-coding a port?
Expected answer: Production environments (cloud platforms, containers) assign ports dynamically via environment variables. The fallback pattern allows the same code to work in development (using the default) and production (using the assigned port).

6. If you define routes in this order: `app.get('*', ...)` then `app.get('/about', ...)`, what happens when a user visits `/about`?
Expected answer: The wildcard `*` route matches first because Express evaluates routes in registration order. The `/about` handler never executes. More specific routes must be defined before catch-all routes.

7. How does `app.get()` behave differently when called with one argument versus two?
Expected answer: With one argument (a string), `app.get(name)` retrieves an application setting. With two arguments (path, handler), it registers a route handler for HTTP GET requests.

Practical Assignment: Build a Book Info API

Objective: Create an Express application that serves both HTML pages and JSON API endpoints, using everything covered in this subtopic.

Instructions:

1. Create a new project directory called `book-info-api`, initialise it with `npm init -y`, and install Express.
2. Create an `app.js` file. Define an array of book objects at the top of the file:

```
const books = [
  { id: 1, title: 'The Pragmatic Programmer', author: 'David Thomas & Andrew Hunt',
year: 1999 },
  { id: 2, title: 'Clean Code', author: 'Robert C. Martin', year: 2008 },
  { id: 3, title: 'Design Patterns', author: 'Gang of Four', year: 1994 }
];
```

3. Create the following routes:
   - `GET /` — Returns an HTML page with a welcome message and links to `/about` and `/api/books`
   - `GET /about` — Returns an HTML page describing the application
   - `GET /api/books` — Returns the full list of books as JSON
   - `GET /api/status` — Returns a JSON object with `status`, `bookCount`, and `uptime` (using `process.uptime()`)
4. Use `process.env.PORT || 3000` for the port.
5. Add a `"start"` script to `package.json`.
6. Test every endpoint using `curl` or a browser.

Expected Output:

```
curl http://localhost:3000/api/books
```

Output:

```
[
  {"id":1,"title":"The Pragmatic Programmer","author":"David Thomas & Andrew Hunt","year":1999},
  {"id":2,"title":"Clean Code","author":"Robert C. Martin","year":2008},
  {"id":3,"title":"Design Patterns","author":"Gang of Four","year":1994}
]
```

```
curl http://localhost:3000/api/status
```

Output:

```
{"status":"running","bookCount":3,"uptime":12.456}
```